

Skill

Task 1 : To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
vennela_07@VennelaPC:~$ mkdir combined_redirection
cd combined_redirection
vennela_07@VennelaPC:~/combined_redirection$ pwd
/home/vennela_07/combined_redirection
vennela_07@VennelaPC:~/combined_redirection$ nano redirection.c
vennela_07@VennelaPC:~/combined_redirection$ echo "This is the input file." > input.txt
vennela_07@VennelaPC:~/combined_redirection$ cat input.txt
This is the input file.
vennela_07@VennelaPC:~/combined_redirection$ gcc redirection.c -o redirection
vennela_07@VennelaPC:~/combined_redirection$ ls
input.txt  redirection  redirection.c
vennela_07@VennelaPC:~/combined_redirection$ ./redirection
vennela_07@VennelaPC:~/combined_redirection$ cat output.txt
Message from standard error
Second stderr message
Message from standard output
Second stdout message
vennela_07@VennelaPC:~/combined_redirection$ nl -ba output.txt
  1 Message from standard error
  2 Second stderr message
  3 Message from standard output
  4 Second stdout message
vennela_07@VennelaPC:~/combined_redirection$ rm input.txt
vennela_07@VennelaPC:~/combined_redirection$ ./redirection
Error opening input.txt: No such file or directory
vennela_07@VennelaPC:~/combined_redirection$ echo "New input data" > input.txt
vennela_07@VennelaPC:~/combined_redirection$ ./redirection
vennela_07@VennelaPC:~/combined_redirection$ cat output.txt
Message from standard error
Second stderr message
Message from standard output
Second stdout message
vennela_07@VennelaPC:~/combined_redirection$ ls -l /proc/$$/fd
total 0
lrwx----- 1 vennela_07 vennela_07 64 Sep 28 02:34 0 -> /dev/pts/0
lrwx----- 1 vennela_07 vennela_07 64 Sep 28 02:34 1 -> /dev/pts/0
lrwx----- 1 vennela_07 vennela_07 64 Sep 28 02:34 2 -> /dev/pts/0
```

```
lrwx----- 1 vennela_07 vennela_07 64 Sep 28 02:34 255 -> /dev/pts/0
vennela_07@VennelaPC:~/combined_redirection$ ls -l
total 28
-rw-r--r-- 1 vennela_07 vennela_07 15 Sep 28 02:34 input.txt
-rw-r--r-- 1 vennela_07 vennela_07 101 Sep 28 02:34 output.txt
-rwxr-xr-x 1 vennela_07 vennela_07 16216 Sep 28 02:33 redirection
-rw-r--r-- 1 vennela_07 vennela_07 1194 Sep 28 02:32 redirection.c
vennela_07@VennelaPC:~/combined_redirection$ cat input.txt
New input data
vennela_07@VennelaPC:~/combined_redirection$ cat output.txt
Message from standard error
Second stderr message
Message from standard output
Second stdout message
```

```
GNU nano 7.2                                redirection.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    int input_fd, output_fd;

    /* Duplicate stdin from input.txt */
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("Error opening input.txt");
        return 1;
    }

    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 stdin failed");
        close(input_fd);
        return 1;
    }

    close(input_fd);

    /* Duplicate stdout to output.txt */
    output_fd = open("output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output_fd < 0)
    {
        perror("Error opening output.txt");
        return 1;
    }

    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 stdout failed");
        close(output_fd);
        return 1;
    }

    close(output_fd);
```

```
    if (dup2(STDOUT_FILENO, STDERR_FILENO) < 0)
    {
        return 1;
    }

    printf("Message from standard output\n");
    fprintf(stderr, "Message from standard error\n");

    printf("Second stdout message\n");
    fprintf(stderr, "Second stderr message\n");

    return 0;
}
```

Task 2 : To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
vennela_07@VennelaPC:~$ cat input.txt | grep Linux > output.txt
cat: input.txt: No such file or directory
vennela_07@VennelaPC:~$ cat input.txt | grep Linux 2> error.txt
cat: input.txt: No such file or directory
vennela_07@VennelaPC:~$ mkdir complex_commands
vennela_07@VennelaPC:~$ mkdir complex_commands
mkdir: cannot create directory 'complex_commands': File exists
vennela_07@VennelaPC:~$ cd complex_commands
vennela_07@VennelaPC:~/complex_commands$ cd complex_commands
-bash: cd: complex_commands: No such file or directory
vennela_07@VennelaPC:~/complex_commands$ nano complex_pipe.c
vennela_07@VennelaPC:~/complex_commands$ #include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* First command: cat input.txt */
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        return 1;
    }

    return 1;
}

if (pid1 == 0)
{
    int fd;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1)
    {
        perror("input.txt");
        exit(1);
    }

    return 0;
}
complex command executed successfully.\n");
vennela_07@VennelaPC:~/complex_commands$ nano input.txt
vennela_07@VennelaPC:~/complex_commands$ ls
complex_pipe.c  input.txt
```

```

vennela_07@VennelaPC:~/complex_commands$ nano input.txt
vennela_07@VennelaPC:~/complex_commands$ ls
complex_pipe.c  input.txt
vennela_07@VennelaPC:~/complex_commands$ gcc complex_pipe.c -o complex_pipe
vennela_07@VennelaPC:~/complex_commands$ ls
complex_pipe  complex_pipe.c  input.txt
vennela_07@VennelaPC:~/complex_commands$ ./complex_pipe
Complex command executed successfully.
vennela_07@VennelaPC:~/complex_commands$ cat output.txt
Linux is an operating system.
Linux supports pipes.
Linux supports redirection.
Ubuntu is based on Linux.
vennela_07@VennelaPC:~/complex_commands$ cat input.txt | grep Linux > output.txt
vennela_07@VennelaPC:~/complex_commands$ cat input.txt | grep Linux > output.txt
vennela_07@VennelaPC:~/complex_commands$ ./complex_pipe
Complex command executed successfully.
vennela_07@VennelaPC:~/complex_commands$ cat output.txt
Linux is an operating system.
Linux supports pipes.
Linux supports redirection.
Ubuntu is based on Linux.
vennela_07@VennelaPC:~/complex_commands$ rm input.txt
vennela_07@VennelaPC:~/complex_commands$ rm input.txt
rm: cannot remove 'input.txt': No such file or directory
vennela_07@VennelaPC:~/complex_commands$ ./complex_pipe
input.txt: No such file or directory
Complex command executed successfully.
vennela_07@VennelaPC:~/complex_commands$ echo "Linux test" > input.txt
vennela_07@VennelaPC:~/complex_commands$ ./complex_pipe
Complex command executed successfully.
vennela_07@VennelaPC:~/complex_commands$ cat output.txt
Linux test

```

```

GNU nano 7.2                                complex_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* First command: cat input.txt */
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        return 1;
    }
}

```

```

}

if (pid1 == 0)
{
    int fd;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1)
    {
        perror("input.txt");
        exit(1);
    }

    /* Input comes from input.txt */
    dup2(fd, STDIN_FILENO);
    close(fd);

    /* Output goes to pipe */
    dup2(pipefd[1], STDOUT_FILENO);

    close(pipefd[0]);
    close(pipefd[1]);

    execlp("cat", "cat", NULL);

    perror("cat");
    exit(1);
}

/* Second command: grep Linux */
pid2 = fork();

if (pid2 == -1)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    int fd;

    /* Input comes from pipe */
    dup2(pipefd[0], STDIN_FILENO);

```

```

        /* Output goes to output.txt */
        fd = open("output.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

        if (fd == -1)
        {
            perror("output.txt");
            exit(1);
        }

        dup2(fd, STDOUT_FILENO);

        close(fd);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("grep", "grep", "Linux", NULL);

        perror("grep");
        exit(1);
    }

    /* Parent closes pipe */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Wait for child processes */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Complex command executed successfully.\n");

    return 0;
}

```

Task 3 : To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
vennela_07@VennelaPC:~$ mkdir background_jobs
vennela_07@VennelaPC:~$ cd background_jobs
vennela_07@VennelaPC:~/background_jobs$ cd background_jobs
-bash: cd: background_jobs: No such file or directory
vennela_07@VennelaPC:~/background_jobs$ nano background_jobs.c
vennela_07@VennelaPC:~/background_jobs$ gcc background_jobs.c -o background_jobs
vennela_07@VennelaPC:~/background_jobs$ ls
background_jobs  background_jobs.c
vennela_07@VennelaPC:~/background_jobs$ ./background_jobs
Simple Background Job Shell
Type 'run' to start a background job
Type 'check' to monitor jobs
Type 'exit' to quit
shell> run
[Parent] Background job launched. PID = 595
[Parent] Prompt returned immediately.
shell> [Child] Background job started. PID = 595
[Child] Background job finished. PID = 595
^C
vennela_07@VennelaPC:~/background_jobs$ sleep(10);
-bash: syntax error near unexpected token `10'
vennela_07@VennelaPC:~/background_jobs$ ./background_jobs
Simple Background Job Shell
Type 'run' to start a background job
Type 'check' to monitor jobs
Type 'exit' to quit
shell> run
[Parent] Background job launched. PID = 598
[Child] Background job started. PID = 598
[Parent] Prompt returned immediately.
shell> sleep(10);
Unknown command.
shell> [Child] Background job finished. PID = 598
check
Background job completed. PID = 598
shell> waitpid(-1, &status, WNOHANG);
Unknown command.
```

```
shell> Unknown command.
shell> Unknown command.
shell> check
No completed background jobs.
shell> run
[Parent] Background job launched. PID = 602
[Child] Background job started. PID = 602
[Parent] Prompt returned immediately.
shell> check
Background job is still running.
shell> [Child] Background job finished. PID = 602
exit
Exiting shell...
```

```
GNU nano 7.2                                     background_jobs.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    pid_t pid;
    int status;

    printf("Simple Background Job Shell\n");
    printf("Type 'run' to start a background job\n");
    printf("Type 'check' to monitor jobs\n");
    printf("Type 'exit' to quit\n");

    while (1)
    {
        char command[20];

        printf("shell> ");
        fflush(stdout);

        scanf("%19s", command);

        /* Exit shell */
        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        /* Start background job */
        else if (strcmp(command, "run") == 0)
        {
            pid = fork();

            if (pid < 0)
            {
                perror("fork failed");
                continue;
            }
        }
    }
}
```

```
        if (pid == 0)
        {
            printf("[Child] Background job started. PID = %d\n",
                getpid());

            sleep(10);

            printf("[Child] Background job finished. PID = %d\n",
                getpid());

            exit(0);
        }

        printf("[Parent] Background job launched. PID = %d\n",
            pid);

        printf("[Parent] Prompt returned immediately.\n");
    }

    /* Check background jobs */
    else if (strcmp(command, "check") == 0)
    {
        pid = waitpid(-1, &status, WNOHANG);

        if (pid == 0)
        {
            printf("Background job is still running.\n");
        }
        else if (pid > 0)
        {
            printf("Background job completed. PID = %d\n", pid);
        }
    }
}
```



```

        printf("Background job completed: PID = %d\n", pid);
    }
    else
    {
        printf("No completed background jobs.\n");
    }
}

else
{
    printf("Unknown command.\n");
}
}

return 0;
}

```

Task 4 : To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```

vennela_07@VennelaPC:~$ mkdir job_management
vennela_07@VennelaPC:~$ cd job_management
vennela_07@VennelaPC:~/job_management$ nano job_table.c
vennela_07@VennelaPC:~/job_management$ gcc job_table.c -o job_table
vennela_07@VennelaPC:~/job_management$ ls
job_table  job_table.c

```

```
vennela_07@VenneLaPC:~/job_management$ ./job_table
Job Management System
Job Management System
```

Commands:

```
run    - Create background job
list   - Display job table
update - Update job status
clean  - Remove completed jobs
exit   - Exit program
```

```
shell> run
Job added successfully.
Background job started with PID 677
```

Commands:

```
[Child] PID = 677 running...
run    - Create background job
list   - Display job table
update - Update job status
clean  - Remove completed jobs
exit   - Exit program
```

```
shell> list
```

Job Table

Job ID	PID	PGID	State
1	677	677	Running

Commands:

```
run    - Create background job
list   - Display job table
update - Update job status
clean  - Remove completed jobs
```

```
shell> run
Job added successfully.
Background job started with PID 678
```

Commands:

```
run    - Create background job
list   - Display job table
update - Update job status
clean  - Remove completed jobs
exit   - Exit program
```

```
shell> [Child] PID = 678 running...
[Child] PID = 677 completed.
[Child] PID = 678 completed.
update
Job statuses updated.
```

Job Table

Job ID	PID	PGID	State
1	677	677	Completed
2	678	678	Completed

Commands:

run - Create background job
list - Display job table
update - Update job status
clean - Remove completed jobs
exit - Exit program

shell> list

Job Table

Job ID	PID	PGID	State
1	677	677	Completed
2	678	678	Completed

Commands:

run - Create background job
list - Display job table
update - Update job status
clean - Remove completed jobs
exit - Exit program

shell> update

Job statuses updated.

Job Table

```
shell> run
Job added successfully.
Background job started with PID 686
```

```
Commands:
run    - Create background job
list   - Display job table
update - Update job status
clean  - Remove completed jobs
exit   - Exit program
```

```
shell> [Child] PID = 686 running...
[Child] PID = 686 completed.
list
```

Job Table

Job ID	PID	PGID	State
3	686	686	Completed

```
Commands:
run    - Create background job
list   - Display job table
update - Update job status
clean  - Remove completed jobs
exit   - Exit program
```

```
shell> exit
Exiting...
Command 'Job' not found, did you mean:
  command 'mob' from snap mob-sh (4.2.0)
  command 'wob' from deb wob (0.14.2-1)
See 'snap info <snapname>' for additional versions.
```

```

GNU nano 7.2                                     job_table.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    pid_t process_group;
    char state[20];
};

struct Job job_table[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

/* Add a job to the table */
void add_job(pid_t pid, pid_t pgid)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    job_table[job_count].job_id = next_job_id++;
    job_table[job_count].pid = pid;
    job_table[job_count].process_group = pgid;

    strcpy(job_table[job_count].state, "Running");

    job_count++;

    printf("Job added successfully.\n");
}

```

```

/* Display job table */
void display_jobs()
{
    int i;

    if (job_count == 0)
    {
        printf("Job table is empty.\n");
        return;
    }

    printf("\nJob Table\n");
    printf("-----\n");
    printf("Job ID\tPID\tPGID\tState\n");
    printf("-----\n");

    for (i = 0; i < job_count; i++)
    {
        printf("%d\t%d\t%d\t%s\n",
            job_table[i].job_id,
            job_table[i].pid,
            job_table[i].process_group,
            job_table[i].state);
    }

    printf("-----\n");
}

```

```

GNU nano 7.2
void update_jobs()
{
    int i;
    int status;
    pid_t result;

    for (i = 0; i < job_count; i++)
    {
        result = waitpid(job_table[i].pid,
                        &status,
                        WNOHANG);

        if (result == 0)
        {
            strcpy(job_table[i].state, "Running");
        }
        else if (result == job_table[i].pid)
        {
            strcpy(job_table[i].state, "Completed");
        }
    }
}

/* Remove completed jobs */
void remove_completed_jobs()
{
    int i, j;

    for (i = 0; i < job_count; i++)
    {
        if (strcmp(job_table[i].state, "Completed") == 0)
        {
            printf("Removing Job ID %d\n",
                    job_table[i].job_id);

            for (j = i; j < job_count - 1; j++)
            {
                job_table[j] = job_table[j + 1];
            }

            job_count--;
            i--;
        }
    }
}

int main()
{
    char command[20];
    pid_t pid;

```

```

int main()
{
    char command[20];
    pid_t pid;

    printf("Job Management System\n");

    while (1)
    {
        printf("\nCommands:\n");
        printf("run    - Create background job\n");
        printf("list   - Display job table\n");
        printf("update - Update job status\n");
        printf("clean  - Remove completed jobs\n");
        printf("exit   - Exit program\n");

        printf("\nshell> ");
        scanf("%19s", command);
    }
}

```

GNU nano 7.2

```
if (strcmp(command, "run") == 0)
{
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        printf("[Child] PID = %d running...\n",
            getpid());

        sleep(10);

        printf("[Child] PID = %d completed.\n",
            getpid());

        exit(0);
    }

    /* Parent */
    add_job(pid, pid);

    printf("Background job started with PID %d\n",
        pid);
}

/* Display jobs */
else if (strcmp(command, "list") == 0)
{
    update_jobs();
    display_jobs();
}

/* Update status */
else if (strcmp(command, "update") == 0)
{
    update_jobs();

    printf("Job statuses updated.\n");

    display_jobs();
}

/* Remove completed jobs */
else if (strcmp(command, "clean") == 0)
{
    update_jobs();
```

```
        printf("Completed jobs removed.\n");
        display_jobs();
    }

    /* Exit */
    else if (strcmp(command, "exit") == 0)
    {
        printf("Exiting...\n");
        break;
    }

    else
    {
        printf("Unknown command.\n");
    }
}

return 0;
}
```


